

Electrical and Computer Engineering Department

ENCS3390 Operating Systems

Assignment 1: Multithreaded Matrix Multiplication with Performance Analysis

Name: Tareq Bashir Subhi Ahmad
NO: 1230453

Name: Bashar Adnan Ghosheh
NO: 1230368

Dr. Yazan Abu Farha

Section: 1

Date: 24/5/2026

Contents

Contents

Contents II
1. Procedures & Discussion 1
1.1 Programming Environment 1
1.2 Test cases: 1
1.2.1 Matrix size 100 x 100 1
1.2.2 Matrix size 500 x 500 2
1.2.3 Matrix size 1000 x 1000 3
1.3 Performance analysis and comparison of approaches: 4
1.4 Speedup compared to the baseline: 5
1.5 Discussion of issues affecting performance: 6
1.5.1 Thread overhead 6
1.5.2 CPU cores 6
1.5.3 Context switching 6
1.5.5 Background processes 6
2. Conclusion 7

1. Procedures & Discussion

1.1 Programming Environment

This project was carried out using a device with the following specifications:

CPU	11th Gen Intel(R) Core(TM) i7-1165G7	4 Cores
		8 logical processors
RAM	16 GB DDR4 Dual Channel	3200 MT/s
OS	Windows 11 pro	x64
Programming Language	C	

Table 1 : Programming environment details

1.2 Test cases:

Several array sizes (100x100, 500x500, 1000x1000) were tested on several different numbers of threads (2 , 8 , 16) five times each.

1.2.1 Matrix size 100 x 100

Threads number	Method	Run 1	Run 2	Run 3	Run 4	Run 5	Average
2	Baseline	0.00427	0.00377	0.00590	0.00606	0.00421	0.004842
	Row wise	0.00336	0.00195	0.00271	0.00224	0.00238	0.002528
	Column wise	0.00254	0.00190	0.00194	0.00191	0.00195	0.002084
	Block (tiled)	0.00440	0.00238	0.00247	0.00284	0.00238	0.002894
8	Baseline	0.00521	0.00411	0.00405	0.00408	0.00381	0.004252
	Row wise	0.00182	0.00206	0.00198	0.00136	0.00114	0.001672
	Column wise	0.00170	0.00113	0.00213	0.00112	0.00108	0.001432

	Block (tiled)	0.00202	0.00242	0.00232	0.00233	0.00158	0.002134
16	Baseline	0.00414	0.00466	0.00383	0.00438	0.00387	0.004176
	Row wise	0.00127	0.00209	0.00136	0.00161	0.00213	0.001692
	Column wise	0.00121	0.00150	0.00125	0.00178	0.00142	0.001432
	Block (tiled)	0.00240	0.00242	0.00290	0.00262	0.00205	0.002478

Table 2 : Test cases for 100 x 100 Matrix multiplication

1.2.2 Matrix size 500 x 500

Threads number	Method	Run 1	Run 2	Run 3	Run 4	Run 5	Average
2	Baseline	0.36257	0.37070	0.44724	0.34636	0.48620	0.402614
	Row wise	0.18834	0.16829	0.23890	0.23223	0.22908	0.211368
	Column wise	0.21482	0.19357	0.21252	0.19988	0.18655	0.201468
	Block (tiled)	0.15149	0.16992	0.20575	0.17983	0.17573	0.176544
8	Baseline	0.31283	0.32694	0.36575	0.35967	0.36788	0.346614
	Row wise	0.09564	0.10054	0.10270	0.12276	0.10065	0.104458
	Column wise	0.09161	0.09677	0.09864	0.11751	0.10350	0.101606
	Block (tiled)	0.08966	0.09711	0.09299	0.10501	0.09662	0.096278
16	Baseline	0.31686	0.37688	0.32390	0.35510	0.37689	0.349926
	Row wise	0.10917	0.10173	0.10035	0.11764	0.10849	0.107476

	Column wise	0.09624	0.09645	0.09558	0.11109	0.09450	0.098772
	Block (tiled)	0.09550	0.09308	0.09816	0.10913	0.09809	0.098792

Table 3 : Test cases for 500 x 500 Matrix multiplication

1.2.3 Matrix size 1000 x 1000

Threads number	Method	Run 1	Run 2	Run 3	Run 4	Run 5	Average
2	Baseline	3.15478	3.72788	3.19450	3.16112	3.22153	3.291962
	Row wise	2.05303	1.67926	1.90056	1.63440	1.50012	1.753474
	Column wise	1.62835	1.53246	1.93047	1.78671	1.40467	1.656532
	Block (tiled)	2.52585	1.28824	1.20430	1.27887	1.27287	1.514026
8	Baseline	3.34602	3.30872	3.46451	3.44801	3.25688	3.364828
	Row wise	1.12810	1.09170	1.08467	1.08703	1.09187	1.096674
	Column wise	0.98185	0.95054	0.96038	0.94480	0.96056	0.959626
	Block (tiled)	0.85202	0.80708	0.80855	0.81310	0.80550	0.81725
16	Baseline	3.49753	3.30375	3.18230	3.26079	3.48187	3.345248
	Row wise	1.04752	1.01308	1.04256	1.00548	1.03898	1.029524
	Column wise	0.83342	0.82869	0.81983	0.82336	0.83167	0.827394
	Block (tiled)	0.82342	0.80865	0.86374	0.81092	0.81165	0.823676

Table 4 : Test cases for 1000 x 1000 Matrix multiplication

1.3 Performance analysis and comparison of approaches:

After we tried several different cases in the test cases, we took the average as shown in the tables above. Relying on taking an average of several values is relatively necessary because performance and execution time change instantaneously due to several reasons, including background processes, programs closing and opening, and the physical condition of the processor, such as temperature and thermal throttling possibilities. The following table represents the final values adopted:

Matrix size	Threads number	Baseline	Row wise	Column wise	Block tiled
100 x 100	2	0.004842	0.002528	0.002084	0.002894
	8	0.004252	0.001672	0.001432	0.002134
	16	0.004176	0.001692	0.001432	0.002478
500 x 500	2	0.402614	0.211368	0.201468	0.176544
	8	0.346614	0.104458	0.101606	0.096278
	16	0.349926	0.107476	0.098772	0.098792
1000 x 1000	2	3.291962	1.753474	1.656532	1.514026
	8	3.364828	1.096674	0.959626	0.81725
	16	3.345248	1.029524	0.827394	0.823676

Table 5 : Average Execution Time (in seconds) for Different Matrix Sizes and Thread Counts

In Block/Tiled Multiplication, we divide the resulting matrix C into small, fixed-size blocks, each measuring 64×64. Instead of each thread taking up entire rows as in Row-wise multiplication or entire columns as in Column-wise multiplication, the resulting matrix is treated as a grid of blocks. The blocks are numbered using a `block_index`, and then distributed to the threads in a fixed manner, with each thread taking a set of blocks according to its `thread_id`. Within each block, the thread counts all C[i][j] elements within its boundaries, using the law of full matrix multiplication. This means that each element is counted by summing `A[i][k]×B[k][j]` for every k values from 0 to n−1. This method ensures that all regions of the resulting matrix C are counted, not just the diagonal blocks. Furthermore, dividing the work into blocks helps improve cache utilization because each thread works on a small, defined area of the resulting array. This method was chosen because it fulfills the project requirement that each thread calculates

a sub-block or tile of the resulting array, specifying the block size and the method of division.

For small arrays like 100x100, execution times were very short, so the difference between methods was limited. However, for larger arrays, especially 500x500 and 1000x1000, the advantage of multiprocessing became more apparent. Row and column processing methods improved performance by splitting the result array into rows or columns. In general, block hashing performed well for larger arrays because it divided the result array into smaller squares, improving data contiguity in cache. Regarding the use of 16 threads, the improvement compared to using 8 threads was minimal, or even worse, because the operating system only recognizes 8 logical processors. Using a larger number of threads creates a context switch state that negatively impacts performance.

1.4 Speedup compared to the baseline:

Speedup was calculated by dividing the baseline execution time by the execution time of each parallel method ($\frac{baseline}{another\ method}$) . A speedup greater than 1 means that the parallel method was faster than the baseline. The largest speedups appeared with larger matrices because the amount of computation was large enough to benefit from multithreading.

Matrix size	Threads number	Row wise speedup	Column wise speedup	Block tiled speedup
100 x 100	2	1.9153x	2.3234x	1.6731x
	8	2.5431x	2.9693x	1.9925x
	16	2.4680x	2.9162x	1.6852x
500 x 500	2	1.9048x	1.9984x	2.2805x
	8	3.3182x	3.4113x	3.6001x
	16	3.2558x	3.5427x	3.5420x
1000 x 1000	2	1.8774x	1.98726x	2.1743x
	8	3.0682x	3.5063x	4.1172x
	16	3.2493x	4.0431x	4.0613x

Table 6 : Speedup of Parallel Matrix Multiplication Methods Compared to Baseline.

The speedup results show that using threads improved performance compared to the baseline method at almost all sizes. At the small 100x100 size, the improvement was

relatively limited because the initial computation time was very small, making the cost of creating and managing threads significant compared to the multiplication time itself. At larger sizes, such as 500x500 and 1000x1000, the effect of parallelism became more pronounced because the number of computations was greater, and thus the threads benefited more from work breakdown. We also observed that increasing the number of threads from 2 to 8 significantly improved performance, but increasing from 8 to 16 did not always yield substantial improvements because the processor has a limited number of cores and logical threads, and exceeding the processor's capacity could lead to overhead due to scheduling and thread switching. The block-tiled method also performed well, especially at large sizes, because it divides the resulting array into smaller blocks, which helps improve cache utilization and reduce the cost of random memory access. Overall, the results show that performance improves with parallelism, but this improvement is not always linear due to factors such as the number of processor cores, the cost of thread management, and the memory access pattern.

1.5 Discussion of issues affecting performance:

Here are some things that may affect performance:

1.5.1 Thread overhead

For small matrix sizes, such as 100×100, the computation time is very short. Therefore, the overhead of creating, scheduling, and synchronizing threads becomes significant compared to the actual multiplication work. This explains why the speedup for small matrices was limited compared to larger matrices.

1.5.2 CPU cores

The CPU used in the experiment has 4 physical cores and 8 logical processors. Because of this, increasing the number of threads beyond the number of logical processors does not guarantee proportional speedup. Extra threads may cause scheduling overhead and context switching.

1.5.3 Context switching

When the number of software threads is larger than the number of logical processors, the operating system must switch between threads. This context switching introduces overhead, which is one reason why using more threads does not always produce better performance.

1.5.5 Background processes

Execution time may vary slightly between runs due to background processes, CPU frequency changes, thermal effects, and Windows scheduling. Therefore, each experiment was run multiple times and the average time was used to obtain more reliable results.

2. Conclusion

In this project, we implemented and compared four matrix multiplication approaches: the baseline single-threaded method, row-wise parallelism, column-wise parallelism, and block-tiled multiplication. The results showed that using multithreading can improve performance, especially when the matrix size becomes larger. For small matrices such as 100×100 times 100×100, the improvement was limited because the computation time was already very small, so the overhead of creating and managing threads had a noticeable effect.

For larger matrices such as 500×500 times 500×500 and 1000×1000 times 1000×1000, the parallel methods performed much better than the baseline. The block-tiled method gave strong results because it divides the result matrix into smaller blocks, which can improve cache usage and memory locality. However, the speedup was not always linear when increasing the number of threads, because the CPU has a limited number of cores and logical processors. Using more threads can also add scheduling and context-switching overhead.

Overall, this project showed that the way we divide the work between threads has a clear effect on performance. Multithreading is useful for large computations, but choosing the right strategy and the right number of threads is important to get good results.